Database Engineering - Unit IV

Functional Dependency, Closure, Minimal Cover, Key Finding, Normal Forms and Decomposition

Coverage

Functional Dependency (FD)
1. Closure
2. Minimal Cover
3. Non-redundant Cover
4. Key Finding: Simple method and Closure method
5. Normal Forms: 1NF, 2NF, 3NF, BCNF
6. Lossy and Lossless Decomposition

Purpose: classroom explanation, semester examination preparation, and practice problems.

1. Functional Dependency (FD)

A functional dependency describes a relationship between attributes in a relation. If the value of attribute set X uniquely determines the value of attribute set Y, we write:

Notation

X → Y
Read as: X functionally determines Y.

Example: STUDENT(RollNo, Name, Dept, Phone). If every RollNo belongs to exactly one student, then:

RollNo → Name, Dept, Phone

This means that once RollNo is known, the corresponding Name, Department and Phone can be determined.

Types of functional dependencies

• Trivial FD: Y is a subset of X. Example: AB → A.

• Non-trivial FD: Y is not a subset of X. Example: A → B.

• Completely non-trivial FD: X and Y have no common attribute. Example: AB → CD.

• Full functional dependency: Y depends on the complete determinant X and not on any proper subset of X.

• Partial dependency: a non-prime attribute depends on only part of a composite key.

• Transitive dependency: X → Y and Y → Z imply that X determines Z indirectly.

Armstrong inference rules

Reflexivity: If Y ⊆ X, then X → Y.
Augmentation: If X → Y, then XZ → YZ.
Transitivity: If X → Y and Y → Z, then X → Z.

2. Closure of an Attribute Set

The closure of an attribute set X, denoted by X^+, is the set of all attributes that can be functionally determined from X using the given functional dependencies.

Why closure is important

Attribute closure is used to:
- identify superkeys and candidate keys
- test whether an FD is implied
- verify redundancy in a set of FDs
- support normalization and decomposition.

Algorithm for finding X^+

- Start with $X^+ = X$.
- For each FD $Y \rightarrow Z$, if Y is contained in X^+, add Z to X^+.
- Repeat until no new attribute can be added.

Worked Example 1

Relation R(A, B, C, D, E) with F = {A → B, B → C, C → D, D → E}. Find A^+.

Step	Closure	Reason
Start	{A}	Initial attribute
1	{A,B}	A → B
2	{A,B,C}	B → C
3	{A,B,C,D}	C → D
4	{A,B,C,D,E}	D → E

Therefore, A^+ = {A,B,C,D,E}. Since A determines all attributes of R, A is a superkey. As A is a single attribute, it is also a candidate key.

Practice

R(A,B,C,D), F = {A → B, B → C, AC → D}. Find A^+ and AC^+.

3. Minimal Cover

A minimal cover is an equivalent set of functional dependencies in which there are no unnecessary attributes and no redundant functional dependencies.

Conditions for a minimal cover

1. Each FD has exactly one attribute on the right-hand side.
2. No attribute on the left-hand side is extraneous.
3. No FD is redundant.

Standard procedure

- Split the right-hand side: A → BC becomes A → B and A → C.
- Remove extraneous attributes from the left-hand side.
- Remove redundant functional dependencies.
- Verify that the resulting set is equivalent to the original set.

Worked Example 1

F = {A → BC, B → C}

Step 1: Split A → BC into A → B and A → C. So F becomes {A → B, A → C, B → C}.

Step 2: A → C is redundant because A → B and B → C already imply A → C.

Minimal cover: {A → B, B → C}.

Worked Example 2

F = {AB → C, A → B}. Check whether B is extraneous in AB → C.

Starting with A, A → B gives B. Now AB → C can be applied, so A determines C. Hence B is extraneous in AB → C.

Replace AB → C by A → C. Therefore the minimal cover is {A → B, A → C}.

4. Non-redundant Cover

A non-redundant cover is a set of functional dependencies in which no entire FD can be removed without changing the closure of the set. It focuses specifically on removing redundant dependencies.

Minimal cover vs non-redundant cover

Non-redundant cover: no FD is redundant.
Minimal cover: no FD is redundant, RHS is single-attribute, and no LHS attribute is extraneous.

Therefore, every minimal cover is non-redundant, but a non-redundant cover need not satisfy all minimal-cover conditions.

Example

F = {A → B, B → C, A → C}

A → C is redundant because A → B and B → C imply A → C. Remove it.

Non-redundant cover = {A → B, B → C}.

Redundancy test

To test whether X → A is redundant, temporarily remove it and compute X^+ using the remaining FDs. If A is still in X^+, then X → A is redundant.

5. Key Finding

A candidate key is a minimal set of attributes that determines every attribute of the relation. Two useful methods are commonly used in examination problems.

Method A: Simple method

A quick strategy is to inspect the left and right sides of the FDs.

Simple method rule

1. Attributes that never appear on the RHS must be present in every candidate key.
2. Start with these mandatory attributes.
3. Add attributes only if the current set cannot determine the full relation.
4. Check minimality.

Example

R(A,B,C,D,E), F = {A → B, B → C, CD → E}

Attributes not appearing on RHS are A and D. Therefore start with AD. From A → B and B → C, AD gives A,B,C,D. Since C and D are now present, CD → E gives E. Thus $(AD)^+$ = {A,B,C,D,E}. Candidate key = AD.

Method B: Closure method

Compute closures systematically and identify minimal attribute sets whose closure equals the full relation.

Example

R(A,B,C,D), F = {A → B, C → D}. A^+ = {A,B}, C^+ = {C,D}, but $(AC)^+$ = {A,B,C,D}. Hence AC is the candidate key.

6. Normalization and Normal Forms

Normalization is the process of organizing data to reduce redundancy and avoid insertion, deletion and update anomalies.

Normalization ladder

UNF → 1NF → 2NF → 3NF → BCNF

Each higher normal form removes a stronger type of dependency-related redundancy.

Common anomalies

• Insertion anomaly: difficulty inserting a fact because another unrelated fact is missing.

• Update anomaly: the same fact occurs in multiple rows and must be updated in several places.

• Deletion anomaly: deleting one fact unintentionally removes another useful fact.

Example relation

ENROLL(StudentID, StudentName, CourseID, CourseName, InstructorID, InstructorName, Grade)

Possible dependencies: StudentID → StudentName; CourseID → CourseName, InstructorID; InstructorID → InstructorName; (StudentID, CourseID) → Grade.

This relation contains repeated student, course and instructor information. Normalization decomposes it into well-structured relations.

6.1 First Normal Form (1NF)

A relation is in 1NF if every attribute contains only atomic, indivisible values and there are no repeating groups or multivalued cells.

Not in 1NF

StudentID	Name	Phone
S01	Amit	9876, 9123

The Phone field stores multiple values.

Converted to 1NF

StudentID	Name	Phone
S01	Amit	9876
S01	Amit	9123

Exam point

1NF mainly deals with atomicity of attribute values. It does not by itself remove partial or transitive dependencies.

6.2 Second Normal Form (2NF)

A relation is in 2NF if it is in 1NF and every non-prime attribute is fully functionally dependent on the whole candidate key. In other words, there must be no partial dependency on part of a composite key.

Example

ENROLL(StudentID, CourseID, StudentName, CourseName, Grade)

Candidate key = (StudentID, CourseID). FDs: StudentID → StudentName; CourseID → CourseName; (StudentID, CourseID) → Grade.

StudentName depends only on StudentID, and CourseName depends only on CourseID. These are partial dependencies, so the relation is not in 2NF.

2NF decomposition

Relation	Attributes
STUDENT	StudentID, StudentName
COURSE	CourseID, CourseName
ENROLL	StudentID, CourseID, Grade

Important

If every candidate key has only one attribute, partial dependency cannot occur. Such a 1NF relation is automatically in 2NF with respect to partial dependency.

6.3 Third Normal Form (3NF)

A relation is in 3NF if it is in 2NF and there is no transitive dependency of a non-prime attribute on a candidate key.

Formal test

For every non-trivial FD X → A, at least one of the following must hold: (1) X is a superkey, or (2) A is a prime attribute.

Example

EMP(EmpID, EmpName, DeptID, DeptName)

FDs: EmpID → EmpName, DeptID and DeptID → DeptName. Therefore EmpID → DeptName transitively through DeptID.

This relation is not in 3NF because DeptName is a non-prime attribute transitively dependent on the key EmpID.

3NF decomposition

EMPLOYEE(EmpID, EmpName, DeptID)
DEPARTMENT(DeptID, DeptName)

Memory rule

2NF removes partial dependency.
3NF removes transitive dependency of non-prime attributes.

6.4 Boyce-Codd Normal Form (BCNF)

A relation is in BCNF if for every non-trivial functional dependency X → Y, X is a superkey.

BCNF rule

Every determinant must be a superkey.

Why BCNF is stronger than 3NF

3NF allows an FD X → A where X is not a superkey if A is prime. BCNF does not allow this exception.

Example: 3NF but not BCNF

R(Student, Course, Instructor) with FDs: (Student, Course) → Instructor and Instructor → Course. Candidate keys are (Student, Course) and (Student, Instructor).

Instructor → Course violates BCNF because Instructor is not a superkey. However, Course is a prime attribute, so the relation can still satisfy 3NF.

Possible BCNF decomposition

R1(Instructor, Course) and R2(Student, Instructor).

Normal-form comparison

Normal Form	Main condition
1NF	Atomic attribute values
2NF	1NF + no partial dependency of non-prime attributes
3NF	2NF + no improper transitive dependency of non-prime attributes
BCNF	For every non-trivial X → Y, X must be a superkey

7. Lossless and Lossy Decomposition

Decomposition divides a relation into smaller relations. A good decomposition should preserve information and preferably preserve dependencies.

Lossless decomposition

A decomposition is lossless if joining the decomposed relations reconstructs exactly the original relation, without generating spurious tuples.

Lossy decomposition

A decomposition is lossy if the natural join of the decomposed relations can produce extra tuples that were not present in the original relation.

Binary lossless-join test

For decomposition of R into R1 and R2, the decomposition is lossless with respect to F if at least one holds:

(R1 ∩ R2) → R1
or
(R1 ∩ R2) → R2

That is, the common attributes must functionally determine all attributes of at least one component.

Lossless example

R(A,B,C), F = {A → B}. Decompose into R1(A,B) and R2(A,C). Common attribute = A. Since A → AB, the common attribute determines R1. Therefore the decomposition is lossless.

Lossy example

R(A,B,C) decomposed into R1(A,B) and R2(B,C), but suppose no FD allows B to determine R1 or R2. The join may create spurious combinations of A and C, so the decomposition may be lossy.

7.1 Worked Decomposition Problem

Let R(A,B,C,D) with F = {A → B, B → C}. Decompose R into R1(A,B,C) and R2(A,D). Determine whether the decomposition is lossless.

Step 1: Find common attributes.

R1 ∩ R2 = {A}.

Step 2: Check whether A determines R1 or R2.

A → B and B → C, so A^+ includes A,B,C. Thus A → ABC, which is R1.

Conclusion: The decomposition is lossless.

Dependency preservation

A decomposition is dependency preserving if all original functional dependencies can be enforced by checking the decomposed relations individually, without joining them.

Design goal

A desirable decomposition is usually:
• lossless
• dependency preserving
• in a suitable normal form such as 3NF or BCNF

8. Combined Examination Problem

Given R(A,B,C,D,E) and F = {A → BC, B → D, CD → E}.

Part A: Find the candidate key

A does not appear on any RHS, so A must be in every key. Compute A^+: A gives B and C; B gives D; with C and D, CD → E gives E. Therefore A^+ = {A,B,C,D,E}. Candidate key = A.

Part B: Find a minimal cover

Split A → BC into A → B and A → C. The set becomes {A → B, A → C, B → D, CD → E}. No LHS attribute is extraneous and no FD is redundant. This is a minimal cover.

Part C: Highest normal form

Since A is a key, A → B and A → C are acceptable in BCNF. B → D violates BCNF because B is not a superkey. It also violates 3NF because D is non-prime. Therefore, assuming atomic values, the relation is in 2NF but not 3NF.

9. Important Practice Problems

Problem 1: R(A,B,C,D), F = {A → B, B → C, AC → D}. Find all candidate keys.

Problem 2: F = {A → BC, B → C, C → D}. Find a minimal cover.

Problem 3: R(A,B,C,D,E), F = {AB → C, C → D, D → E}. Find the candidate key.

Problem 4: F = {AB → C, A → B, C → D}. Find a minimal cover.

Problem 5: R(StudentID, CourseID, StudentName, CourseName, Grade). Identify the highest normal form if StudentID → StudentName, CourseID → CourseName, and (StudentID,CourseID) → Grade.

Problem 6: R(EmpID, DeptID, DeptName), F = {EmpID → DeptID, DeptID → DeptName}. Identify the normal form violation and decompose to 3NF.

Problem 7: R(A,B,C) with F = {A → B}. Test whether decomposition R1(A,B), R2(A,C) is lossless.

Problem 8: R(A,B,C) decomposed into R1(A,B), R2(B,C), with no useful FD on B. Explain why the decomposition may be lossy.

Short answer key

1. Candidate key A. 2. {A → B, B → C, C → D}. 3. Candidate key AB. 4. {A → B, A → C, C → D}. 5. Not in 2NF because of partial dependencies. 6. Transitive dependency, decompose EMP(EmpID,DeptID) and DEPT(DeptID,DeptName). 7. Lossless. 8. Common attribute B does not determine either component, so spurious tuples may appear.

10. Quick Revision Sheet

Topic	Remember
FD	X → Y means X determines Y
Closure	X+ gives all attributes derivable from X
Candidate key	Minimal superkey
Minimal cover	Single RHS, no extraneous LHS, no redundant FD
Non-redundant cover	No complete FD can be removed
1NF	Atomic values
2NF	No partial dependency
3NF	No improper transitive dependency of non-prime attributes
BCNF	Every determinant is a superkey
Lossless decomposition	Join reconstructs exactly the original relation
Lossy decomposition	Join may create spurious tuples

Final exam strategy

For FD problems, first identify keys using closure. For minimal cover, split RHS first. For normalization, identify the key and then check partial and transitive dependencies. For decomposition, always test the common attributes for the lossless-join condition.